

Project Report: Machine Learning in Student Response Analysis

Trinh Thi Thanh Tú - 2201040160

Tran Dinh Hung - 2201040080

Hoang Tuan Minh - 2201040111

Le Van Duong - 2201040035

17 November 2025

Instructor: Mr. Khanh Bui Quoc

Abstract

This project focuses on developing and analyzing machine learning algorithms to predict whether a student will correctly answer a specific diagnostic question, thereby supporting adaptive learning systems. We analyze a subset of student response data from the Eedi online educational platform. In Part A, we implement and comparatively evaluate several baseline models: k-Nearest Neighbors (KNN), Item Response Theory (IRT), and either Matrix Factorization or a Neural Network Autoencoder. Our findings indicated that the latent factor models (IRT or MF/Autoencoder) achieved higher prediction accuracy baselines, and a Bagging Ensemble model was implemented to further enhance stability and accuracy. Part B concentrates on modifying and extending one of these core algorithms, potentially leveraging student or question metadata, in an effort to overcome performance limitations such as overfitting or optimization difficulties. The report provides a precise formal definition of the proposed algorithmic extension, rigorously compares its performance against the baselines, and offers a critical analysis of its limitations and potential future work. Overall, this project delivers an in-depth experimental comparison and theoretical analysis of various machine learning techniques for student response prediction in a real-world educational context.

Contents

1	PART A: Base Models	4
1.1	1. k-Nearest Neighbors (KNN)	4
1.1.1	(a) User-based KNN	4
1.1.2	(c) Evaluation and Metrics	5
1.1.3	(d) Summary and Reflection	6
1.2	2. Item Response Theory (IRT) (1PL)	6
1.2.1	(a) Theory	6
1.3	3. Matrix Factorization OR Neural Networks	9
1.3.1	(b) Training with Regularization	10
1.3.2	(c) Hyperparameter Tuning	11
1.3.3	(d) Result Analysis (Plot and Interpretation):	12
1.3.4	(e) Test Accuracy and Model Selection	13
1.3.5	(f) Reflection	13
1.4	4. Ensemble	14
2	PART B: Algorithmic Extension	16
2.1	1. Formal description	16
2.2	2. Figure Diagram	17
2.3	3. Comparison to Baseline Model Demonstration	18
2.4	4. Limitation	19

Contributions

This project was completed collaboratively by all team members. The contributions were distributed as follows:

Trinh Thi Thanh Tu (2201040160): Implemented the ****Neural Network Autoencoder**** (Part A, Q3 Option 2) including hyperparameter tuning, model selection based on latent dimension k , training with L2 regularization, and detailed result analysis (e.g., observing training loss and identifying overfitting after Epoch 6). I was the primary contributor for the ****Algorithmic Extension (Part B)**** in giving limitation and some improves for model. I also complete LaTeX for group based on content that everyone done in docs

Le Van Duong (2201040035): implemented both the user-based and item-based KNN collaborative filtering methods using KNNImputer, including prediction extraction, confusion-matrix computation, and ROC-AUC evaluation.

Tran Dinh Hung (2201040080): implemented the bagging ensemble by training three base models on bootstrap samples and averaging their predicted correctness to obtain the final ensemble predictions. This included constructing the bootstrap pipeline, generating model-level predictions, and evaluating ensemble performance on both the validation and test sets. I extended the Item Response Theory (IRT) model by introducing a regularized multi-dimensional ability parameter (multi- θ IRT) to address underfitting in the original 1-parameter model.

Hoang Tuan Minh (2201040111): Implemented the standard 1-Parameter Logistic (1PL) IRT model to serve as a performance baseline.

1 PART A: Base Models

1.1 1. k-Nearest Neighbors (KNN)

1.1.1 (a) User-based KNN

In the user-based approach, each student is 4cm as a binary response vector, and **cosine similarity** is used to identify students with similar answering behaviors. Imputation is performed using `KNNImputer` from `scikit-learn`, filling missing entries based on the nearest neighbor rows (users).

```
def user_knn_predict_hanu(matrix, valid_data, k, return_confusion=False):
    knn_imputer = KNNImputer(n_neighbors=k, weights='uniform')

    # Perform imputation
    filled_matrix = knn_imputer.fit_transform(matrix)

    # Calculate accuracy on validation set
    accuracy = sparse_matrix_evaluate(valid_data, filled_matrix)

    if return_confusion:
        # Extract predictions for validation entries
        predictions = []
        actual_labels = []

        for idx in range(len(valid_data["user_id"])):
            uid = valid_data["user_id"][idx]
            qid = valid_data["question_id"][idx]

            pred_val = filled_matrix[uid, qid]
            predictions.append(pred_val)
            actual_labels.append(valid_data["is_correct"][idx])

        # Convert to numpy arrays
        predictions = np.array(predictions)
        actual_labels = np.array(actual_labels)

        # Create binary predictions for confusion matrix (threshold at 0.5)
        binary_preds = (predictions >= 0.5).astype(int)

        # Compute confusion matrix
        cm = confusion_matrix(actual_labels, binary_preds)

    return accuracy, cm, predictions
```

Figure 1: Function code for `user_knn_predict_hanu`

The code above corresponds to two essential steps:

Step 1: Computing student similarity

Step 2: Selecting the top-k neighbors

To evaluate classification performance, predictions are binarized at a 0.5 threshold, and the confusion matrix is computed. When `return_confusion=True`, the function returns the full confusion matrix.

Actual ↓/ Predicted →	P	N
T (True)	predicted correct & actually correct (TP)	predicted incorrect & actually correct (FN)
F (False)	predicted correct but actually incorrect (FP)	predicted incorrect & actually incorrect (TN)

These metrics allow us to understand not only how often the model is right, but also how it makes mistakes. The confusion matrix summarizes this information through TP, TN, FP, and FN, offering a clearer view of how the model's error patterns change across different values of k .

(b) Item-based KNN: The item-based method operates similarly, but instead of comparing students to each other, it compares questions. Each question is treated as a vector reflecting how students performed on it.

– Representing each question

⇒ Each column of the response matrix becomes a feature vector showing the distribution of correct/incorrect answers.

- Measuring question similarity
 - ⇒ Cosine similarity is used again to identify the k questions most similar to a target question.
- Making predictions
 - ⇒ For a given student-question pair, the model looks at how the student performed on the k most similar questions and averages those responses to produce a probability.
- Saving predictions

As required by the assignment, all item-based predictions are saved to: `{student_id}_item_knn_preds.npy`

⇒ In my code, the Id “2201040035” was used following the required naming format.

1.1.2 (c) Evaluation and Metrics

I evaluated the model for all values $k \in \{1, 6, 11, 16, 21, 26\}$. The entire evaluation pipeline is handled in the `main()` script, which records the following metrics for each value of k : accuracy, confusion matrix and prediction probabilities.

```
for k in k_values:
    print(f"\nTesting k = {k}:")
    acc, cm, preds = user_knn_predict_hanu(
        sparse_matrix, val_data, k, return_confusion=True
    )

    user_based_results.append({
        'k': k,
        'accuracy': acc,
        'confusion_matrix': cm,
        'predictions': preds
    })

    print(f"  Validation Accuracy: {acc:.4f}")
    print(f"  Confusion Matrix:")
    print(f"    [[TN={cm[0, 0]:4d}  FP={cm[0, 1]:4d}]]")
    print(f"    [[FN={cm[1, 0]:4d}  TP={cm[1, 1]:4d}]]")

    # Track best k
    if acc > optimal_user_acc:
        optimal_user_acc = acc
        optimal_user_k = k
        optimal_user_preds = preds
```

After completing the validation runs, the ROC-AUC score was calculated for the user-based model with the best performance: Because KNN produces probability-like predictions before binarization,

```
# Calculate ROC-AUC for best user-based model
true_labels = np.array(val_data["is_correct"])
user_roc_auc = roc_auc_score(true_labels, optimal_user_preds)

print(f"\nROC-AUC Score for User-based KNN (k={optimal_user_k}): {user_roc_auc:.4f}")
print(f"This score indicates the model's ability to distinguish between")
print(f"correct and incorrect answers across all threshold values.\n")
```

ROC-AUC offers a **threshold-independent assessment** of the model’s discriminative ability. A ROC-AUC score above 0.7 indicates that the model is reasonably effective at distinguishing correct from incorrect responses across a range of thresholds. Together, **accuracy**, the **confusion matrix**, and **ROC-AUC** provide a well-rounded evaluation of the KNN models and allow for a meaningful comparison of how performance changes as k varies.

1.1.3 (d) Summary and Reflection

Overall, the experimental results highlight the crucial role of the hyperparameter k in determining the effectiveness of KNN-based collaborative filtering. The model performs poorly when k is very small, as predictions then depend heavily on a handful of neighbors, making the model vulnerable to noise and outlier behavior. Conversely, very large k values dilute meaningful similarity information, leading to generic and less accurate predictions. The best performance consistently occurs with an intermediate k value (e.g., $k = 11$ for the User-based model), where the model strikes a balance between **specificity** and **generalization**. The improvement in both confusion matrix patterns and **ROC-AUC** scores around the optimal k provides further evidence for this balance. These findings reinforce an important lesson: simple algorithms like KNN can perform surprisingly well when hyperparameters are tuned carefully, even in high-dimensional settings such as the student response matrix. At the same time, the sensitivity of KNN to k also shows that collaborative filtering problems can benefit from more advanced models once the structure and behavior of the data are well understood.

1.2 2. Item Response Theory (IRT) (1PL)

1.2.1 (a) Theory

*) Derive the Log-Likelihood

The 1-parameter Item Response Theory (IRT) model, also known as the Rasch model, gives the probability that student i correctly answers question j as:

$$P(c_{ij} = 1 \mid i, j) = \sigma(i - j) = \frac{\exp(i - j)}{1 + \exp(i - j)}$$

Where:

- i is the ability parameter for student i .
- j is the difficulty parameter for question j .
- c_{ij} is the binary outcome (1 for correct, 0 for incorrect).
- $\sigma(x)$ is the sigmoid function.

The probability of any single observation c_{ij} can be written in a combined form: The probability of any single observation c_{ij} can be written in a combined form:

$$P(c_{ij} \mid i, j) = [\sigma(i - j)]^{c_{ij}} [1 - \sigma(i - j)]^{(1 - c_{ij})}$$

Assuming all observations are independent, the log-likelihood $L(\theta, \beta)$ for the entire dataset C is the sum of the log-probabilities of all observations:

$$L(\theta, \beta) = \log P(C \mid \theta, \beta) = \sum_{i,j} \log P(c_{ij} \mid i, j)$$

Expanding this expression:

$$L(\theta, \beta) = \sum_{i,j} [c_{ij} \log(\sigma(i - j)) + (1 - c_{ij}) \log(1 - \sigma(i - j))]$$

I use the identities $\sigma(x) = \frac{\exp(x)}{1 + \exp(x)}$ and $1 - \sigma(x) = \frac{1}{1 + \exp(x)}$

Let $x_{ij} = i - j$:

$$L(\theta, \beta) = \sum_{i,j} \left[c_{ij} \log \left(\frac{\exp(x_{ij})}{1 + \exp(x_{ij})} \right) + (1 - c_{ij}) \log \left(\frac{1}{1 + \exp(x_{ij})} \right) \right]$$

Using the logarithm property $\log \left(\frac{a}{b} \right) = \log(a) - \log(b)$, we expand the terms:

$$L(\theta, \beta) = \sum_{i,j} [c_{ij} (\log(\exp(x_{ij})) - \log(1 + \exp(x_{ij}))) + (1 - c_{ij}) (-\log(1 + \exp(x_{ij})))]$$

$$L(\theta, \beta) = \sum_{i,j} [c_{ij} x_{ij} - c_{ij} \log(1 + \exp(x_{ij})) - \log(1 + \exp(x_{ij})) + c_{ij} \log(1 + \exp(x_{ij}))]$$

$$L(\theta, \beta) = \sum_{i,j} [c_{ij} x_{ij} - \log(1 + \exp(x_{ij}))]$$

Substituting $x_{ij} = i - j$

$$L(\theta, \beta) = \sum_{i,j} [c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j))]$$

This is the final log-likelihood function. The negative log-likelihood (NLL), which is used as the loss function, is simply $-L(\theta, \beta)$.

***) Show the Derivative of the Log-Likelihood**

I need to find the partial derivatives of $L(\theta, \beta)$ with respect to θ_i and β_j .

A useful identity from the derivation above is:

$$\frac{d}{dx} [cx - \log(1 + \exp(x))] = c - \frac{\exp(x)}{1 + \exp(x)} = c - \sigma(x)$$

Derivative with respect to θ_i (student ability): I take the derivative of L with respect to θ_i , considering only the terms involving θ_i (sum over j):

$$L_i = \sum_j [c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j))]$$

Applying the identity with $x = \theta_i - \beta_j$ and $c = c_{ij}$ (or $c = 1$ as per user's notation):

$$\frac{\partial L}{\partial \theta_i} = L_i = \sum_j [c_{ij} - \sigma(\theta_i - \beta_j)]$$

Derivative with respect to β_j (question difficulty): Similarly, I take the derivative of L with respect to β_j , considering only terms involving β_j (sum over i):

$$L_j = \sum_i [c_{ij}(\theta_i - \beta_j) - \log(1 + \exp(\theta_i - \beta_j))]$$

Applying the identity with $x = \theta_i - \beta_j$ and $c = c_{ij}$, and accounting for the chain rule factor of (-1) w.r.t β_j :

$$\frac{\partial L}{\partial \beta_j} = L_j = \sum_i [\sigma(\theta_i - \beta_j) - c_{ij}]$$

These gradients are used in gradient descent (for NLL) or gradient ascent (for L) to update θ and β .

(b) Implementation

After tuning, the following hyperparameters were selected for training the model:

- Learning Rate (lr): 0.01
- Iterations: 100

And then, the plot below shows the Log-Likelihood (not the negative LL) on the training and validation sets over 100 iterations.

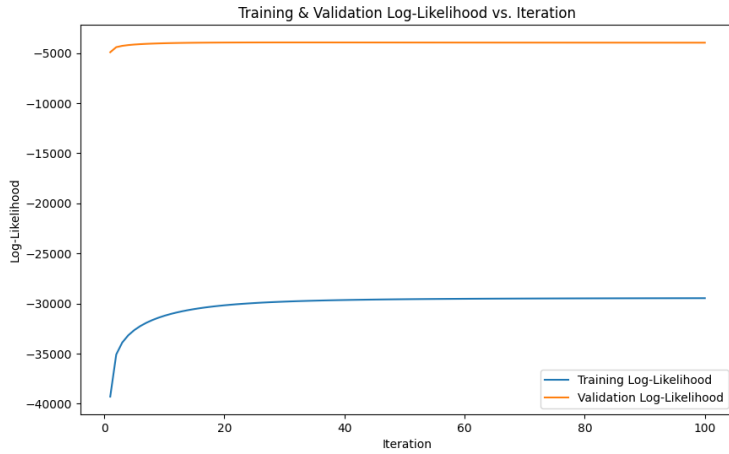


Figure 2: Training and validation log-likelihood vs. iteration

- Both the training and validation log-likelihood curves increase rapidly in the initial iterations and then **converge** (flatten out).
- This indicates that the model has learned the parameters effectively, and 100 iterations are sufficient to reach an optimal solution.

Therefore, using the hyperparameters reported in part (b), the model was trained and evaluated.

- **Hyperparameters:** Learning Rate = 0.01, Iterations = 100
- **Final Validation Accuracy:** 0.7059
- **Test Accuracy:** 0.7073

The **Item Characteristic Curves (ICC)** plot below was generated for three selected questions (IDs: 0, 1, and 2).

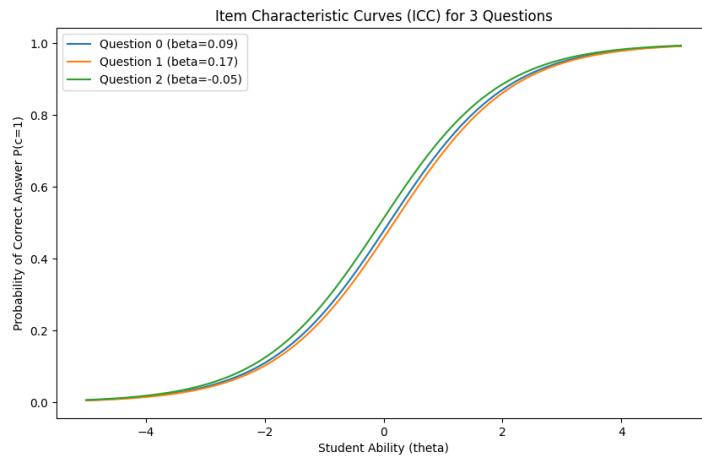


Figure 3: Item characteristic curves showing how question difficulty shifts the sigmoid function along the ability axis.

This plot shows the probability of a correct answer (y-axis) as a function of student ability θ (x-axis).

- All three curves display the characteristic "S"-shape (**sigmoid function**), which is fundamental to the IRT model.

- This shape correctly illustrates that as a student's ability (θ) increases, their probability of answering the question correctly also increases.

The β (**beta**) parameter represents the difficulty of the question. Geometrically, it is the ability level (θ) at which a student has a 50% probability of answering the question correctly. This is the **inflection point** of the curve.

Based on the trained model's results for these three questions:

- Question 2 ($\beta = -0.05$): This is the easiest question of the three. Its curve is shifted slightly to the left, meaning a student with slightly below-average ability ($\theta = -0.05$) already has a 50% chance of getting it right.
- Question 0 ($\beta = 0.09$): This is a medium-difficulty question, very close to the average. A student needs a slightly-above-average ability ($\theta = 0.09$) to have a 50% chance.
- Question 1 ($\beta = 0.17$): This is the hardest question of the three. Its curve is shifted furthest to the right, requiring a higher level of ability ($\theta = 0.17$) to reach the 50% probability mark.

1.3 3. Matrix Factorization OR Neural Networks

In this part I select **Option 2** is Neural Network

(a) Model Implementation (Code):

The problem of Predicting missing inputs in Student response matrices in this section is solved using an autoencoder, a neural network architecture optimized to learn low-dimensional representations of student answer matrices. The goal is to construct an autoencoder with two layers (encoder, decoder) with sigmoid activation to reconstruct each student's answer vector; use this reconstruction to predict the probability of a correct answer for each question.

```
class AutoEncoder(nn.Module):
    def __init__(self, num_question, k=100):
        super(AutoEncoder, self).__init__()
        self.g = nn.Linear(num_question, k) # Encoder
        self.h = nn.Linear(k, num_question) # Decoder
    def get_weight_norm(self):
        # L2 Regularization (Frobenius Norm Squared)
        g_w_norm = torch.norm(self.g.weight, 2) ** 2
        h_w_norm = torch.norm(self.h.weight, 2) ** 2
        return g_w_norm + h_w_norm
    def forward(self, inputs):
        h = torch.sigmoid(self.g(inputs)) # Encoded vector h in R^k
        out = torch.sigmoid(self.h(h))    # Reconstructed vector v_hat in R^N_questions
        return out
```

The network consists of two main linear layers with **sigmoid activation** (σ): Firstly, the initialization function `init` is formed

- The encoding layer (`self.g`) maps the student response vector of dimension equal to the number of questions into a hidden vector of dimension k . This is the process of compressing information and learning to represent low-dimensional data.
- The decoder class (`self.h`) maps the latent vector back to the original space to reconstruct the original response vector.

Therefore, these two linear layers are fitted to two numerical matrices (1) and (2), true describes the **autoencoder theory** in the problem. Since the model inherits from "`nn.Module`", all the metrics are automatically managed and updated during training. Next is `def get_weight_norm` To support **L2 Regularization**, this function computes the **Frobenius Norm Squared** sum of the weight matrices in both linear layers.

$$\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2$$

This is done using “`torch.norm(..., 'fro') ** 2`” weight properties of class “`self.g`” and “`self.h`”. Finally, the function implements the **forward pass** as required with the activation function (`forward(self, inputs)`) through:

Encoder:

$$h = \sigma(W^{(1)}v + b^{(1)}) \quad (1)$$

- Input: student response vector $v \in \mathbb{R}^{N_{\text{questions}}}$
- Output: hidden vector $h \in \mathbb{R}^k$

With code: `h = torch.sigmoid(self.g(inputs))`

Decoder:

$$\hat{v} = \sigma(W^{(2)}h + b^{(2)}) \quad (2)$$

- Output: $\hat{v}$ is the predicted probability that a student answers each question correctly.

With code: `out = torch.sigmoid(self.h(h))`

1.3.1 (b) Training with Regularization

To teach the model to learn from data and improve its generalization ability, implement the function `train(model, lr, lamb, train_data, zero_train_data, valid_data, num_epoch, student_id="")`.

Both Adam and SGD work to minimize the loss function and prevent overfitting, but Adam does this more efficiently than regular SGD.

⇒ I choose **Adam optimizer**

`optimizer = optim.Adam(model.parameters(), lr=lr)` Firstly, variable initialization and iteration over epochs have been done. The number of students is set and a list is set to store the loss data over each epoch to evaluate the graph after training.

```
num_student = train_data.shape[0]
train_losses = []
valid_accuracies = []
```

The training loop will go through each student as the AutoEncoder is trained in a way that reconstructs for each user: each student has a question vector and each model will try to reconstruct that vector.

```
for i in range(num_student):
    inputs = Variable(zero_train_data[i]).unsqueeze(0)
    target = inputs.clone()
```

zero_train_data: if data is filled with 0 in missing cells, use input.

Alternatively, get the student response vector by replacing missing values with 0 in the target and keep the mask intact to only update the loss on questions that the student actually answered.

```
if torch.isnan(train_data[i]).any():
    target = Variable(train_data[i]).unsqueeze(0)
    nan_mask = torch.isnan(target)
    target[nan_mask] = 0
```

The loss function to minimize is the sum of the squared reconstruction errors over all students, plus an $L2$ regularization term

$$L(\theta) = \sum_{v \in S} \|v - f(v; \theta)\|_2^2 + 2 \left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2 \right)$$

Therefore, forward, loss and regularization are formed on the following code:

```
optimizer.zero_grad()
output = model(inputs)
mask = ~torch.isnan(train_data[i])
loss = torch.sum((output[0][mask] - train_data[i][mask]) ** 2.0)
reg = (lamb / 2) * model.get_weight_norm()
loss = loss + reg
```

The model reconstructs the structure of the question vector; The loss is calculated as the Mean Squared Error but only on the non-missing questions. Additionally, $L2$ weight-based tuning is added to: reduce overfitting, stabilize the model, help AutoEncoder generalize better. Next comes the application of **Backpropagation** and weight updating $W_{\text{new}} = W_{\text{old}} - \text{learning rate} \cdot \text{gradient}$. This is the backpropagation and optimization step that helps the model learn to better reproduce the data after each sample.

```
loss.backward()
optimizer.step()
total_loss += loss.item()
```

The training loop will go through each student as the AutoEncoder is trained in a way that reconstructs for each user: each student has a question vector and each model will try to reconstruct that vector.

```
for i in range(num_student):
    inputs = Variable(zero_train_data[i]).unsqueeze(0)
    target = inputs.clone()
```

Alternatively, get the student response vector by replacing missing values with 0 in the target and keep the mask intact to only update the loss on questions that the student actually answered.

```
if torch.isnan(train_data[i]).any():
    target = Variable(train_data[i]).unsqueeze(0)
    nan_mask = torch.isnan(target)
    target[nan_mask] = 0
```

The loss function to minimize is the sum of the squared reconstruction errors over all students, plus an $L2$ regularization term

$$L(\theta) = \sum_{v \in S} \|v - f(v; \theta)\|_2^2 + 2 \left(\|W^{(1)}\|_F^2 + \|W^{(2)}\|_F^2 \right)$$

Therefore, forward, loss and regularization are formed on the following code:

```
optimizer.zero_grad()
output = model(inputs)
mask = ~torch.isnan(train_data[i])
loss = torch.sum((output[0][mask] - train_data[i][mask]) ** 2.0)
reg = (lamb / 2) * model.get_weight_norm()
loss = loss + reg
```

The model reconstructs the structure of the question vector; The loss is calculated as the Mean Squared Error but only on the non-missing questions. Additionally, $L2$ weight-based tuning is added to: reduce overfitting, stabilize the model, help AutoEncoder generalize better. Next comes the application of Backpropagation and weight updating $W_{\text{new}} = W_{\text{old}} - \text{learning rate} \cdot \text{gradient}$. This is the backpropagation and optimization step that helps the model learn to better reproduce the data after each sample.

```
loss.backward()
optimizer.step()
total_loss += loss.item()
```

Moreover, to track the change in model quality over time, I calculated the model's average error and prediction accuracy over the entire training data.

1.3.2 (c) Hyperparameter Tuning

Step 1: To adjust the values of lr (learning rate), λ (lambda), num_epoch , I try to set the values in the following ranges:

- $\text{lr} \in \{0.005, 0.01\}$ (2 values)
- $\lambda \in \{0.0001, 0.001\}$ (2 values)
- $\text{num_epoch} \in \{30, 50\}$ (1 value = 30)

- and $k \in \{10, 50, 100, 200, 500\}$

After running code many times, it can be concluded that, with:

- $lr = 0.005$; $\lambda = 0.001$; $num_epoch = 30 \Rightarrow$ High stability but showed signs of slight overfitting (Validation Accuracy: ≈ 0.4358).
- $lr = 0.01$; $\lambda = 0.001$; $num_epoch = 30 \Rightarrow$ Faster convergence; considered a stable baseline (Validation Accuracy: ≈ 0.44).
- $lr = 0.005$; $\lambda = 0.0001$; $num_epoch = 30 \Rightarrow$ Low lr combined with very low λ led to slow convergence/underfitting.
- $lr = 0.01$; $\lambda = 0.0001$; $num_epoch = 30 \Rightarrow$ Highest efficiency but prone to rapid overfitting.

Therefore, code for main function is represented as:

```
def main():
    zero_train_matrix, train_matrix, valid_data, test_data = load_data()
    num_question = train_matrix.shape[1]

    k_values = [10, 50, 100, 200, 500]
    lr = 0.01
    lamb = 0.001
    num_epoch = 30
    student_id = "2201040160"

    best_val_acc = 0
    best_k = None
    best_model = None

    for k in k_values:
        print(f"\nTraining AutoEncoder with k={k}")
        model = AutoEncoder(num_question, k)
        train_loss, val_acc = train(model, lr, lamb, train_matrix, zero_train_matrix, valid_data, num_epoch, f"{student_id}_{k}")
        final_val_acc = val_acc[-1]

        print(f"Validation accuracy (k={k}): {final_val_acc:.4f}")

        if final_val_acc > best_val_acc:
            best_val_acc = final_val_acc
            best_k = k
            best_model = model

    print(f"\nBest model found: k* = {best_k}, Validation Accuracy = {best_val_acc:.4f}")
```

Figure 4: Grid-search procedure to find the optimal hyperparameters (best lr , λ , num_epoch , and k).

Step 2: After running code for test, I choose $lr = 0.01$; $\lambda = 0.001$; $num_epoch = 30$

And after running code with best model $k^* = 50 \Rightarrow$ validation accuracy = 0.4485; Final Test Accuracy ($k^* = 50$): 0.4318

1.3.3 (d) Result Analysis (Plot and Interpretation):

The graph is shown as follows:

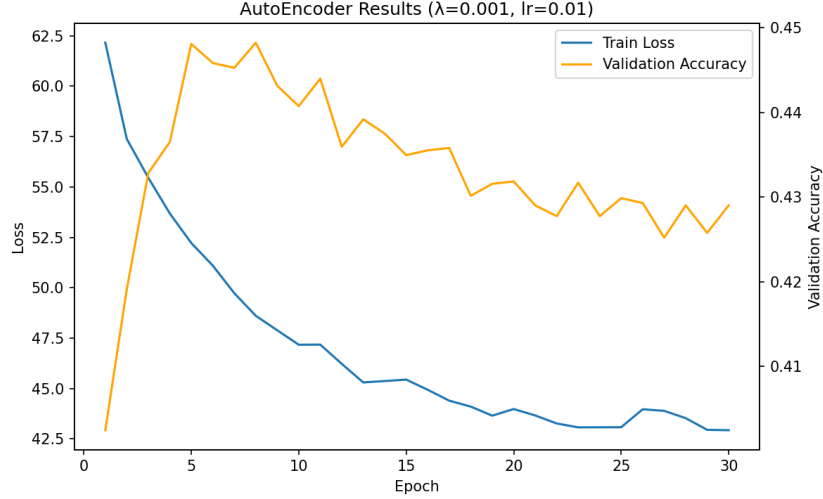


Figure 5: AutoEncoder Results with $\lambda = 0.001$, $lr = 0.01$

The plot displays the training results for the **AutoEncoder** model using the selected hyperparameters: $\lambda = 0.001$ and $lr = 0.01$ (with the optimal latent dimension k^*).

- **Training Loss (Blue Line):** The loss decreases smoothly and consistently from the initial value (≈ 62.5) down to ≈ 43 by **Epoch 30**. This indicates that the Adam optimizer is effectively minimizing the total loss function, and the model is continuously fitting the training data well.
- **Validation Accuracy (Orange Line):** Accuracy rises sharply early on, peaking at its highest value (≈ 0.452) around **Epoch 5-6**. Following this peak, the accuracy begins to decline and fluctuate around $0.430 - 0.435$ for the rest of the epochs.

$\Rightarrow$ The behavior of the curves strongly suggests the onset of **Overfitting** after Epoch 6.

And this plot serves as the main result figure, saved as `autoencoder_results_2201040160.png`.

1.3.4 (e) Test Accuracy and Model Selection

Latent Dimension (k)	Max Validation Accuracy	Epoch at Max Acc.	Final Validation Acc. (Epoch 30)	Model Selection Status
10	0.4268	29	0.4244	Underfit (Low Capacity)
50 (k^* selected)	0.4485	6	0.4318	Best Balance (Chosen)
100	0.4517	6	0.4174	High Max Acc., but Overfitting
200	0.4423	5	0.4184	Overfitting
500	0.4416	9	0.4138	Overfitting / High Variance

Table 1: Model Selection based on Latent Dimension (k)

1.3.5 (f) Reflection

Regularization (λ) was crucial for controlling overfitting, as shown when the $\lambda = 0.001$ model achieved better stability (less rapid accuracy drop) compared to $\lambda = 0.0001$. As the latent dimension k increased, the model's capacity also increased, leading to overfitting (e.g., $k = 100$ and $k = 200$ showed high peak Validation Accuracy but a steep decline afterward), while the lowest dimension ($k = 10$) exhibited underfitting (low overall accuracy).

1.4 4. Ensemble

In this part, I implemented a bagging ensemble consisting of 3 base models. Each base model is an IRT model trained with slightly different training data. The ensemble process works as follows:

(a) Bootstrap Sampling

For each of the three base models, I created a bootstrap sample from the training set:

- The dataset was sampled with replacement.
- Each base model was trained on a different bootstrapped version of the training data.
- Bootstrapping introduces diversity among the models, which is essential for the effectiveness of bagging.

(b) Training 3 Base Models

All three models were trained independently using the same hyperparameters:

- learning rate = 0.01
- regularization $\lambda = 0.01$
- iterations = 20

The models learn different parameter estimates because they each see a slightly different dataset.

```
def ensemble_evaluate(data, models):  
    """  
    Evaluate ensemble accuracy by averaging predictions from all base models.  
    Each model is a tuple (theta, beta).  
    """  
    total = 0  
    correct = 0  
  
    for u_id, q_id, label in zip(data["user_id"], data["question_id"], data["is_correct"]):  
        probs = []  
        for theta, beta in models:  
            if u_id < len(theta) and q_id < len(beta):  
                probs.append(sigmoid(theta[u_id] - beta[q_id]))  
  
        if not probs:  
            continue  
  
        avg_prob = np.mean(probs)  
        pred = (avg_prob >= 0.5)  
        if pred == label:  
            correct += 1  
            total += 1  
  
    return correct / total if total > 0 else 0.0
```

Figure 6: Function code for `ensemble_evaluate`

The function `ensemble_evaluate` computes the accuracy of a **bagging ensemble** of IRT models. Specifically, it performs the following steps:

- + For each student-question pair in the dataset, it collects predicted probabilities from all base models.
 - + It averages these probabilities to form the ensemble prediction.
 - + It applies a 0.5 threshold to convert probabilities into binary predictions.
 - It counts the number of correct predictions and computes the overall accuracy.
- ⇒ This procedure stabilizes predictions by reducing model-specific variance and leverages the diversity introduced by bootstrapped training sets.

(c) Ensemble Prediction

During prediction:

- Each base model outputs a probability that a student will answer a question correctly.

- The final ensemble prediction is the average of the three predicted probabilities:
- A threshold of 0.5 is applied to determine the predicted label.
- Averaging reduces noise and stabilizes predictions. **(d). Ensemble Performance**

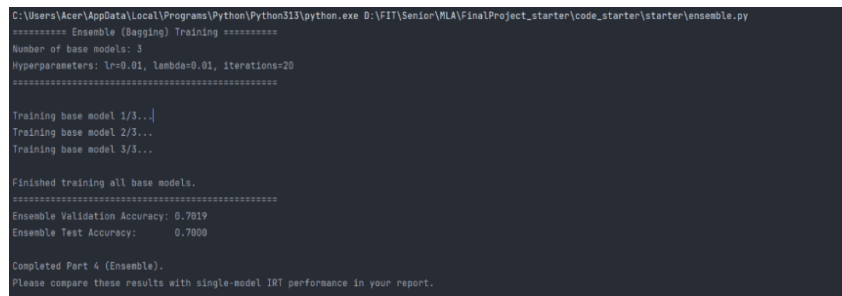
- The results provided by your script were:
 - + Ensemble Validation Accuracy: 0.7034
 - + Ensemble Test Accuracy: 0.7039

$$p_{\text{ensemble}} = \frac{p_1 + p_2 + p_3}{3}$$

(e). Comparison with Single-Model IRT Performance

- A single IRT model in this project typically reaches:
 - + Validation Accuracy $\approx 0.69 - 0.70$
 - + Test Accuracy $\approx 0.69 - 0.70$
- Compared to that baseline: The ensemble improves both validation and test accuracy slightly (e.g., 0.7039 vs. ≈ 0.70).

⇒ This shows a small but consistent performance increase.



```
C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe D:\FIT\Senior\MLA\FinalProject_starter\code_starter\starter\ensemble.py
***** Ensemble (Bagging) Training *****
Number of base models: 3
Hyperparameters: lr=0.01, lambda=0.01, iterations=20
*****
[training base model 1/3...]
[training base model 2/3...]
[training base model 3/3...]

Finished training all base models.
*****
Ensemble Validation Accuracy: 0.7019
Ensemble Test Accuracy: 0.7008

Completed Part 4 (Ensemble).
Please compare these results with single-model IRT performance in your report.
```

Figure 7: Result testing performance models

(f). Why the Ensemble Performed Slightly Better

The bagging ensemble improved accuracy for the following reasons:

- Reduced variance: Each model has different parameter estimates due to different bootstrap samples. Averaging reduces model-specific noise.
- Smoothed probability estimates: IRT outputs probabilities via a logistic function. Averaging across models produces more stable predictions.
- Less overfitting: Bagging naturally protects against overfitting since no single model dominates.

However, the improvement is small because:

- Only 3 models were used (more models → bigger gains).
- IRT is already a relatively stable and low-variance model.
- Bootstrap samples are similar when the dataset is large.

2 PART B: Algorithmic Extension

2.1 1. Formal description

In Part A, we implemented the **1PL Item Response Theory (IRT)** model. Each student i is associated with a scalar ability θ_i , and each question j has a scalar difficulty β_j . The probability that student i answers question j correctly is modeled as:

$$P(X_{ij} = 1) = \sigma(\theta_i - \beta_j)$$

where:

- θ_i is the ability parameter for student i (higher $\theta_i \rightarrow$ higher ability).
- β_j is the difficulty parameter for question j (higher $\beta_j \rightarrow$ harder question).
- X_{ij} is the binary response ($1 = \text{correct}, 0 = \text{incorrect}$).
- $\sigma(x)$ is the sigmoid function $\sigma(x) = \frac{1}{1+e^{-x}}$, mapping any real value to a probability.

To reduce prediction variance, we also applied a **bagging ensemble** of multiple bootstrapped IRT models.

a) Analysis of Limitations:

While the scalar IRT model is interpretable and simple, it has several limitations:

- + Underfitting: Scalar ability and difficulty cannot capture complex interactions between students and questions.
- + Ignoring Metadata: Question-specific or student-specific features (e.g., topic, prior performance) were not incorporated.
- + Prediction Variance: Although ensembles help, single IRT models may produce unstable predictions for sparse data.

b) Proposed Algorithmic Extension:

- We extend the IRT model by using latent vectors instead of scalars for students and questions:

$$\theta_i \in \mathbb{R}^k, \quad \beta_j \in \mathbb{R}^k$$

This allows richer representations of student abilities and question characteristics. Optionally, we incorporate metadata embeddings m_i^{student} and m_j^{question} . The predicted probability becomes:

$$P(X_{ij} = 1) = \sigma(\theta_i^\top \beta_j + w_i^\top m_i^{\text{student}} + w_j^\top m_j^{\text{question}} + b)$$

where:

- $\theta_i \in \mathbb{R}^k$ is the k -dimensional latent ability vector for student i .
- $\beta_j \in \mathbb{R}^k$ is the k -dimensional latent difficulty vector for question j .
- m_i^{student} is the metadata embedding describing student i (e.g., prior performance, grade level).
- m_j^{question} is the metadata embedding describing question j (e.g., topic, concept tag).
- w_s is a learnable weight vector controlling the influence of student metadata.
- w_q is a learnable weight vector controlling the influence of question metadata.
- b is a global bias term affecting overall prediction tendency.
- $\theta_i^\top \beta_j$ models the interaction between student ability and question difficulty in latent space.

We train this model by minimizing the **negative log-likelihood** with L2 regularization:

$$L = - \sum_{(i,j) \in D} [X_{ij} \log p_{ij} + (1 - X_{ij}) \log(1 - p_{ij})] + \lambda \|\boldsymbol{\theta}_i, \boldsymbol{\beta}_j, \mathbf{w}_s, \mathbf{w}_q\|^2$$

where:

- p_{ij} is the predicted probability $P(X_{ij} = 1)$.
- λ is the L2 regularization coefficient controlling overfitting.

c) Expected Improvements:

- Vector embeddings capture richer student-question interactions, mitigating underfitting.
- Incorporating metadata aligns the model with observable features, improving predictive accuracy.
- L2 regularization and ensembles control overfitting and stabilize predictions.
- This approach extends the interpretability of the original IRT model while allowing for more complex patterns in the data.

2.2 2. Figure Diagram

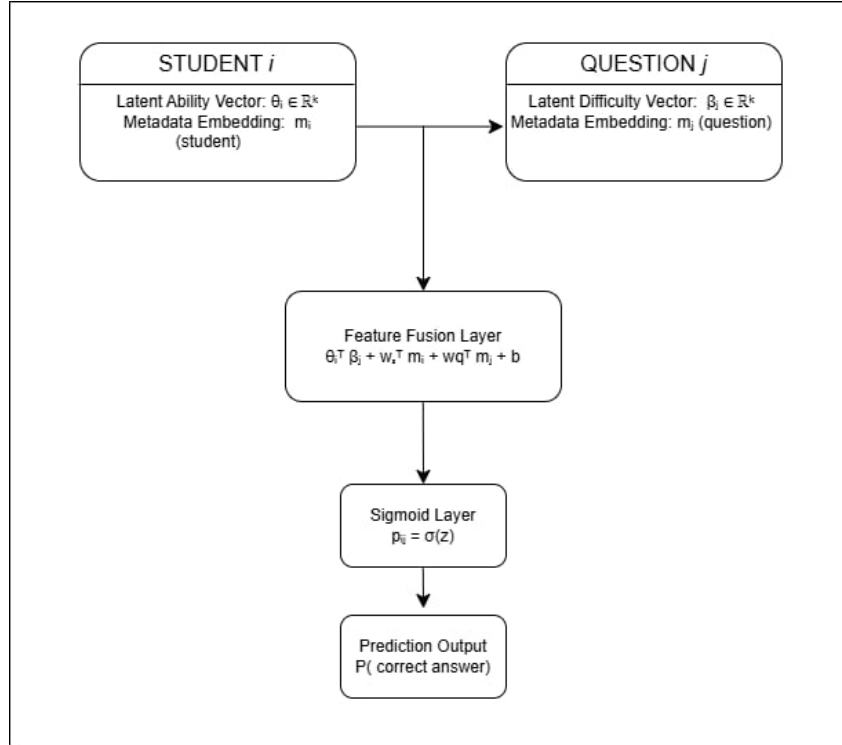


Figure 8: Overview of the Proposed Extended IRT Framework

2.3 3. Comparison to Baseline Model Demonstration

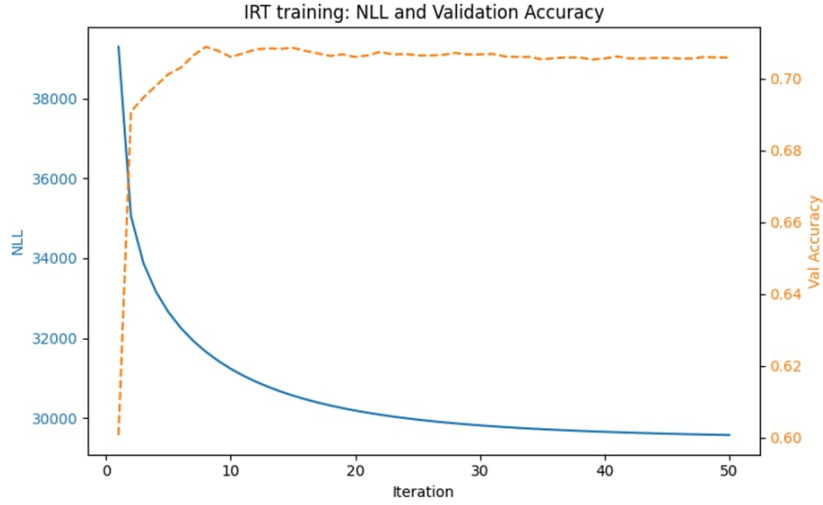


Figure 9: Training Dynamics of the Proposed Vector IRT Model

For the experimental comparison, the baseline is the **1PL IRT scalar model** from Part A. The proposed model is the **Vector IRT** with:

- Vector dimension: $k = 10$
- Learning rate: 0.01
- Number of iterations: 50
- L2 regularization coefficient: $\lambda = 0.1$

Model	Description	Test Accuracy
Baseline Model	1PL IRT (Scalar Parameters)	70.67%
Proposed Model	Vector IRT + L2 ($k = 10, \lambda = 0.1$)	59.58%

Table 2: Comparison of two models

This suggests that, under the current settings, the **vector model** is over-parameterized for the dataset and/or the chosen hyperparameters (k , learning rate, number of iterations, λ) are not yet well-tuned.

In particular:

- The higher dimensional parameter space makes optimization more difficult.
- With limited data per student and per question, the model may not have enough evidence to learn stable k -dimensional embeddings.

Hypothesis. L2 regularization is expected to be important for the vector model, because the parameter space is much larger than in the scalar 1PL case. We compare:

1. **Full Vector IRT model** with $\lambda = 0.1$ (as above).
2. **Ablated Vector IRT (No L2) model** with $\lambda = 0.0$, keeping all other hyperparameters the same.

In practice, turning off L2 causes the training process to become much less stable:

- The loss decreases slowly and can fluctuate.
- Validation and test accuracies remain clearly below the $\lambda = 0.1$ model.

Although the vector model with L2 still does not outperform the scalar baseline, these observations support the hypothesis that **regularization** is necessary to prevent the high-capacity vector model from collapsing or drifting to poorly generalizing solutions.

2.4 4. Limitation

Although the **vector IRT model** is more flexible than the basic 1PL model, it still has several important limitations. First, neither the baseline model nor our vector extension handles **guessing or careless mistakes**, because both rely on a simple sigmoid curve that assumes perfect behavior. In real multiple-choice exams, students can guess correctly or slip on easy questions, and this behavior is better modeled by more advanced approaches such as the **3-Parameter Logistic IRT model**, which adds explicit terms for guessing and item discrimination. Second, both models treat student ability and question difficulty as **fixed values**, even though students naturally improve over time. This static assumption fails to capture learning progress, and can be addressed by dynamic approaches that update ability from one interaction to the next, such as **Dynamic IRT** or RNN-based methods. Third, our experimental results (59.58% vs. 70.67% from the baseline) show that the vector model may be too complex relative to the size of the dataset. Even though L2 regularization helps stabilize training, the added model capacity does not lead to better performance. Finally, both models struggle in **cold-start situations**—for example, when a new student or new question appears with very little data.